

企业级仓储物流管理平台的设计与实现

系别： 计算机科学与技术系

班级： 计 23-2 班

姓名： 周凯君

学号： 23101020222

2025 年 12 月 31 日

目录

1 课题主要内容介绍

1.1 课题概述

本课题旨在模拟真实企业的开发流程，设计并构建一套“企业级仓储物流管理系统”。系统采用前后端分离架构，前端使用 Vue 3 构建响应式用户界面，后端使用 Node.js (Express) 提供 RESTful API 接口，并连接 SQL Server 数据库进行持久化存储。系统主要功能包括：

- **用户鉴权：**基于数据库验证的管理员登录与退出，UI 包含头像与姓名展示。
- **工作台 (Dashboard)：**展示物流数据的统计概览，左上角标识系统名称。
- **包裹管理：**实现对物流包裹的增、删、改、查 (CRUD) 全流程管理。
- **状态追踪：**实时更新包裹位置与运输状态（待发货、运输中、已送达）。

1.2 运行环境说明

- 操作系统：Windows 11 64 位
- 开发工具：Visual Studio Code, SQL Server Management Studio (SSMS)
- 前端技术栈：Vue 3 (Composition API) + Vite + Element Plus
- 后端技术栈：Node.js + Express
- 数据库：Microsoft SQL Server (使用 Windows 身份验证)
- 驱动程序：msnodesqlv8 (原生 ODBC 驱动)

2 系统设计与实现

2.1 系统分析与设计

在企业仓储管理中，核心需求是数据的实时性与准确性。设计目标如下：

1. **界面规范：**采用侧边栏布局，左侧为功能导航，左上角显示“仓储管理系统”Logo，右上角显示当前登录用户的头像与姓名。
2. **数据安全：**所有敏感操作（如删除包裹）需经过后端数据库验证，且必须登录后才能操作。
3. **交互体验：**使用 Element Plus 组件库，提供弹窗表单、气泡确认等友好交互。

2.2 系统模块设计

系统分为以下三层架构：

- **表示层 (Frontend):** ‘Login.vue’ (登录页)、‘Layout’ (主布局框架)、‘Packages’ (业务列表页)。
- **逻辑层 (Backend):** ‘index.js’ 处理路由分发与 SQL 语句拼接，处理前端发来的 JSON 数据。
- **数据层 (Database):** SQL Server 中的 ‘Users’ 表 (存储管理员信息) 和 ‘Packages’ 表 (存储物流信息)。

2.3 系统实现

2.3.1 确定系统实现思路

为了实现企业级的高效开发，我选择了“全栈 JavaScript”方案。前端 Vue 3 通过 Axios 发送异步请求，后端 Express 接收请求后，通过 ‘mssql/msnodesqlv8’ 驱动调用本地 SQL Server 数据库，实现真正的前后端联调。

2.3.2 后端数据库连接 (难点突破)

由于本地 SQL Server 使用 Windows 身份验证，常规驱动无法连接。我引入了 ‘msnodesqlv8’ 驱动并配置了连接字符串，成功解决了权限问题。

```
1 const express = require('express');
2 const sql = require('mssql/msnodesqlv8'); // 引入原生驱动
3
4 // 数据库配置：指定 ODBC Driver 17
5 const dbConfig = {
6   connectionString: 'Driver={ODBC Driver 17 for SQL Server};Server=
   localhost;Database=WarehouseDB;Trusted_Connection=yes;'
7 };
8
9 // 连接数据库
10 sql.connect(dbConfig).then(() => {
11   console.log('SQL Server Connected Successfully');
12 }).catch(err => console.error('Connection Failed:', err));
```

Listing 1: Node.js 连接 SQL Server 核心代码

2.3.3 前端页面构建

前端使用 ‘App.vue’ 结合 ‘v-if’ 指令实现单页应用 (SPA) 的路由切换效果。左侧导航栏支持折叠与展开，右侧内容区域动态加载。

```
1 // 获取包裹列表
2 const fetchData = async () => {
3   try {
4     const res = await axios.get('http://localhost:3000/api/packages');
5     tableData.value = res.data;
6   } catch (error) {
7     ElMessage.error('无法连接后端服务');
8   }
9 }
10
11 // 删除包裹
12 const handleDelete = (id) => {
13   ElMessageBox.confirm('确定删除该包裹吗?', '警告').then(async () => {
14     await axios.delete(`http://localhost:3000/api/packages/${id}`);
15     ElMessage.success('已删除');
16     fetchData(); // 刷新列表
17   });
18 }
```

Listing 2: 前端 API 请求示例 (Axios)

3 页面效果展示

3.1 登录界面

系统提供安全的登录入口，需输入管理员账号与密码。

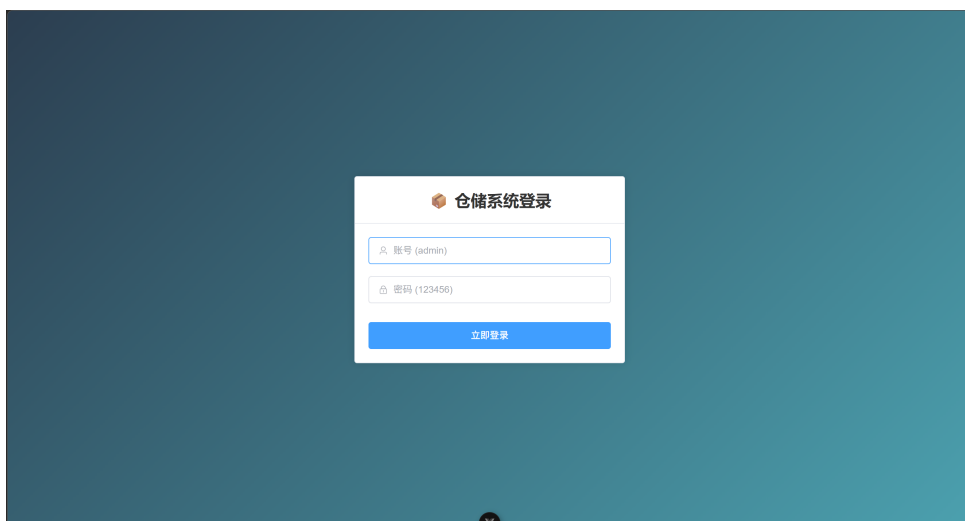


图 1: 系统登录页

3.2 系统工作台

登录成功后进入工作台，左侧为导航，右上方展示用户信息。



图 2: 管理后台工作台

3.3 包裹管理与新增

支持通过弹窗新增包裹，并在表格中实时显示物流状态。

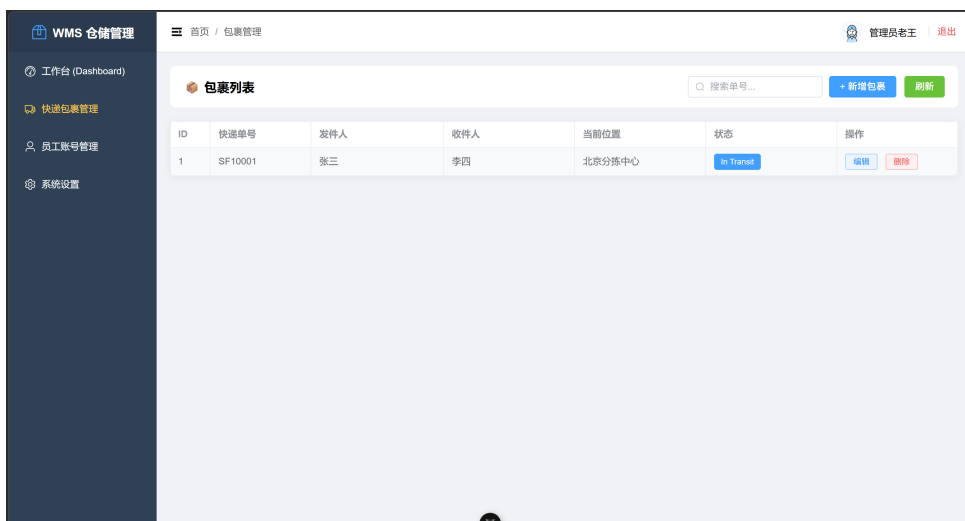


图 3: 增删改查功能演示

4 系统测试分析

1. **连通性测试:** 后端控制台显示 ‘Connected’, 且前端能从数据库拉取到 ‘Users’ 表中的 ‘admin’ 账号信息, 登录跳转功能正常。
2. **CRUD 业务测试:**
 - **增:** 点击 “新增包裹”, 填写表单后, 数据库 ‘Packages’ 表增加一行记录。
 - **删:** 点击 “删除” 按钮, 前端列表行消失, 数据库记录同步被物理删除。
 - **改:** 更新状态为 “已送达”, 前端标签颜色变为绿色, 状态同步成功。
3. **异常测试:** 关闭后端服务时, 前端弹出 “无法连接后端” 的红色错误提示, 系统健壮性符合预期。

5 课题收获总结

5.1 企业级项目构建体会

通过本次课程设计, 我从零搭建了一个全栈 WMS 系统。最大的体会在于 “规范性”: 使用 Element Plus 让我明白统一 UI 风格的重要性; 使用 SQL Server 让我体会到了关系型数据库在企业数据存储中的核心地位。

5.2 解决问题的感悟

开发过程中最大的困难是 Node.js 连接本地 SQL Server 时的证书与驱动问题。通过查阅文档, 我学会了使用 ‘msnodesqlv8’ 配合 ‘ODBC Driver 17’ 来解决 Windows 身

份验证问题。这让我明白，全栈开发不仅是写代码，更是环境配置与调试能力的考验。

6 参考文献

- [1] Vue.js 3.0 官方文档. <https://vuejs.org/>
- [2] Element Plus 组件库指南. <https://element-plus.org/>
- [3] Microsoft SQL Server 连接驱动文档. <https://www.npmjs.com/package/mssql>